Computer Vision - CSOC Report

Name: Sushant Kumar
Roll No: 25165058
Branch: Pharmaceutical Engineering

Task 0: Custom CNN Implementation

For Task 0, I first processed the images into a $128 \times 128 \times 3$ format. I found this to be a sweet spot; a 224×224 resolution would be comparatively large, and I wanted to keep my model small. Specifically, if I had made it deeper, I would have had to include a ResNet-like architecture, which I wanted to avoid. Therefore, I had to keep the model shallow, making $128 \times 128 \times 3$ a good fit in my judgment.

Firstly, I imported the necessary libraries and extracted the data. In the Caltech-256 dataset, there are 257 classes but only 80–90 images per class, which is too few for a model to train properly. To address this, I applied data augmentation, which helped increase the accuracy a bit. Techniques like small-angle rotations and random horizontal flips ensured that the model emphasized the right features to classify and didn't memorize incorrect patterns. Since I was training a CNN, I utilized a GPU for the heavy computations.

Architecture Details

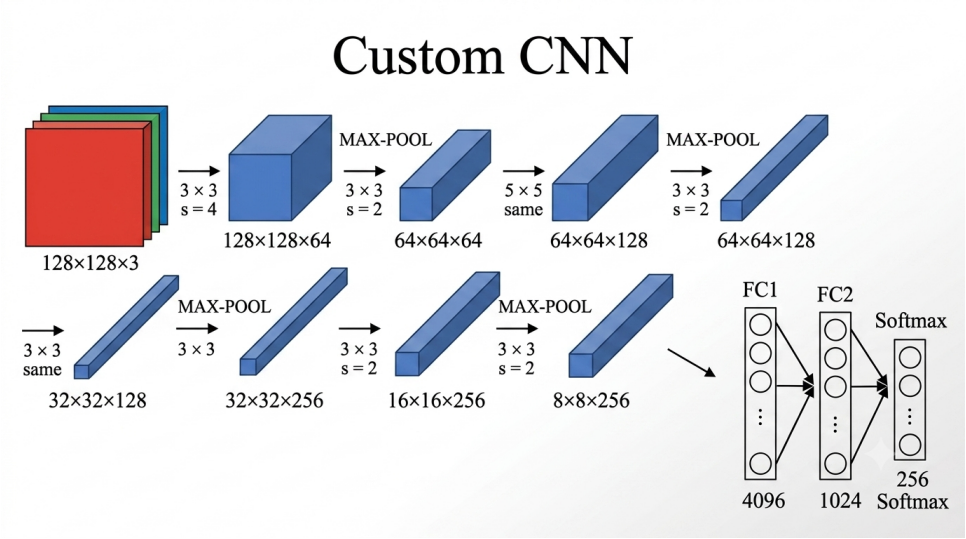

Figure 1: Grad-CAM visualization for a correctly classified sample.

Let's break the architecture down. Firstly, I implemented convolutional layers with filters having a kernel size of 3×3, rather than 5×5 or 7×7 like in AlexNet. The reasoning behind this is simple:

- A stack of two 3×3 convolutional layers has an effective receptive field of 5×5. Three such layers have a 7×7 effective receptive field.
- Incorporating three non-linear rectification layers instead of a single one makes the decision function more discriminative.
- It decreases the number of parameters: a stack of three 3×3 layers is parameterized by 27 weights, while a single 7×7 conv layer would require 49 parameters.

Initially, I implemented two convolutional layers back-to-back. The thought process here was to extract minute details, like edges, first and combine them to form larger features that hold enough significance before applying max pooling. Max pooling helps extract the maximum significance from a collection of weights representing a broader feature, while simultaneously reducing the parameters.

Why did I keep the padding as "same" and use a stride of 1? Every pixel in the feature map, including border pixels, gets to be the center of a convolution operation. Without padding, border pixels are visited far fewer times than center pixels, meaning information at the edges is underutilized. "Same" padding ensures every spatial location contributes equally to feature extraction. Since I already kept my neural network comparatively shallow, I had the intuition to use "same" padding to maximize information retention. The same reasoning for the stride being 2 is basically : to extract maximum of learning while still reducing the weights in the parameters by a significant amount since I already decided to keep the model small.

I placed batch normalization after every layer. This improves accuracy because, after several layers of computation, different weights may scale differently, making normalization crucial. Dropouts were added in the fully connected (FC) layers to reduce overfitting, which is a common practice. Weight decay (regularization) was also implemented for the same reason.

The reasoning behind the FC layers is straightforward. The image size was reduced to $8 \times 8 \times 256$, but we cannot pass that many parameters directly to the logits layer and ask it to classify into 257 classes. I could have used Global Average Pooling (GAP) here, but I chose to go with FC layers, as GAP would make the already shallow CNN even shallower. The parameters were decreased gradually: $8 \times 8 \times 256 \rightarrow 4096 \rightarrow 1024 \rightarrow 257$.

Initially, I set the epochs to 10 because the computational time was high. When I ran it, the CNN honestly worked perfectly, and the loss decreased steadily. In only 10 epochs, I achieved 22% accuracy, indicating that increasing the epochs would easily push it to around 50%. Later, I ran it for about 50 epochs, and the accuracy reached 40%.

Task 1: ResNet-18 Implementation

After VGG-16, ResNet-18 was introduced. The researchers behind ResNet noticed an unusual curve when plotting model depth against accuracy. According to theory, as network depth increases, model accuracy should also increase. However, eventually (after approximately 20+ layers), the accuracy started to degrade. To solve this, the researchers introduced the concept of residual connections (hence the name ResNet).

A residual connection is a network mechanism where the information or output from a preceding layer is passed directly to a subsequent layer, skipping over intermediate layers. This shortcut essentially solves a few main problems:

- When a network is extremely deep, there may be some layers where no learning occurs. A standard plain network with ReLU activations cannot easily pass a pure identity function. ResNets solve this by allowing the architecture to directly pass the identity function if a layer learns nothing significant.
- It helps mitigate the vanishing gradient problem when the network gets too deep.

Furthermore, if a layer receives both the transformed data and the identity shortcut,

$$F(x) + x$$

, it learns what to *change* or refine in the existing input, rather than computing an entirely new transformation from scratch. The accuracy of my self coded ResNet18 finally came out to be 50.16% after running it for 50 epochs.

Task 2: Grad-CAM Analysis

For the third part, we will understand Grad-CAM. First, let's explore the mathematics and reasoning behind the Grad-CAM formula.

Let Y^c represent the value (or score) for a specific class c in the final logit layer before the softmax function. What we are essentially doing is taking the derivative of Y^c with respect to the feature map activations A^k of the last convolutional layer.

Why the last convolutional layer, and why a derivative? In a CNN, the final convolutional layer retains the most abstract and high-level spatial characteristics of an image. When we compute the derivative $\frac{\partial Y^c}{\partial A^k}$, we are finding out how much the predicted class score Y^c changes when the activations in a specific feature map A^k are adjusted. Since this layer contains the largest features, it is the most preferable candidate for visual explanations.

This gradient effectively gives us the "importance" of each feature map A^k. We term this spatially pooled gradient (or importance weight) as α_k^c:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial Y^c}{\partial A_{i,j}^k}$$

What we do next is multiply the importance of each feature map (α_k^c) by the feature map itself (A^k) and take a weighted sum. We apply a ReLU activation to only highlight the regions that have a positive influence on the class of interest:

$$L_{Grad-CAM}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right)$$

This weighted sum generates a heatmap that increases parameters where the map contributes more to classifying that specific class, and decreases parameters where the map contributes less. The model focuses on the regions it used to reason its prediction the most. An incorrect prediction simply tells us that the model gave an incorrect output plainly due to focusing heavily on the wrong region.

Honestly, I chose Grad-CAM as the specific interpretability method because it does not require any architectural changes to my pretrained model for its implementation—unlike other CAM methods that strictly require adding a Global Average Pooling (GAP) layer.